

МИНОБРНАУКИ РОССИИ
САНКТ-ПЕТЕРБУРГСКИЙ ГОСУДАРСТВЕННЫЙ
ЭЛЕКТРОТЕХНИЧЕСКИЙ УНИВЕРСИТЕТ
«ЛЭТИ» ИМ. В.И. УЛЬЯНОВА (ЛЕНИНА)
Кафедра МО ЭВМ

ОТЧЕТ

по индивидуальному домашнему заданию №2
по дисциплине «Распределенные алгоритмы»

Тема: Моделирование и верификация алгоритма Барца в среде SPIN

Студенты гр. 3343

Бондаренко Ф.А.

Пименов П.В.

Преподаватель

Шошмина И.В.

Санкт-Петербург

2026

Задание.

Вариант 6

- Реализовать задачу доступа процессов в критическую секцию с помощью алгоритма Барца, симулирующего обобщенный семафор с помощью бинарных.
- Необходимо смоделировать алгоритм на языке Promela и верифицировать в среде SPIN по сформулированным свойствам (LTL).

Выполнение работы.

1. Параметры модели и состав переменных

Глобальные переменные:

```
byte S = 1; // бинарный мьютекс для count
byte gate = 1; // бинарный турникет на входе
int count = K; // счетчик свободных слотов (= k - #вКС)
byte critical = 0; // датчик присутствия в КС
```

Первые три — собственно алгоритм Барца. *critical* — не часть алгоритма: служебный счетчик, увеличивается на входе в КС и уменьшается на выходе. Через него LTL-свойство $ltl\ mutex\ \{ []\ (critical \leq K) \}$ отслеживает факт нарушения взаимного исключения.

Тип *byte* для *S* и *gate* — намеренно, а не *bit/bool*: LTL должен уметь наблюдать значение > 1 , если бы алгоритм был некорректен. С типом *bit* свойство $[]\ (gate \leq 1)$ стало бы очевидным, и контрпример невозможно было бы найти.

2. Реализация семафорных операций

В Promela нет встроенного типа «семафор», но *busy-wait* семафор (с активным ожиданием) выражается макросами *wait/signal*:

```
#define wait(s)    atomic { s > 0; s-- }
#define signal(s) s++
```

atomic обеспечивает атомарность операции (обязательное свойство семафора), а голое булево выражение $s > 0$ — это исполнимая инструкция Promela, блокирующая процесс, пока выражение ложно.

Тонкость атомарности: внутри *atomic*, если инструкция блокируется ($s > 0$ при $s = 0$), атомарность временно снимается. Другой процесс может выполнить *signal(s)*, поднять *s* до 1, и наш *wait* продолжится с *s--* уже атомарно — поскольку $s > 0$ стало исполнимым.

3. Протокол входа в КС

Соответствует шагам p1-p6 листинга 6.13 у Бен-Ари:

```
wait(gate); // p1: пройти турникет
wait(S); // p2: захватить мьютекс на count
count = count - 1; // p3: занять слот
if
:: count > 0 -> signal(gate); // p4, p5
:: else      -> skip;
fi;
signal(S); // p6: отпустить мьютекс
```

Важная часть — ветка *else* в *if/fi*. При $count=0$ $signal(gate)$ не выполняется, $gate$ остается 0, и следующий процесс блокируется на $wait(gate)$. Так турникет закрывает вход, когда все K слотов заняты.

4. Протокол выхода из КС

Шаги p7-p11, симметрично входу:

```
wait(S); // p7: захватить мьютекс
count = count + 1; // p8: освободить слот
if
:: count == 1 -> signal(gate); // p9, p10:
:: else      -> skip;
fi;
signal(S); // p11: отпустить мьютекс
```

Условие $count=1$ принципиально: $signal(gate)$ срабатывает только при переходе $count:0 \rightarrow 1$ (был занят последний слот, теперь освободился). Это обеспечивает баланс — на каждое «закрытие» турникета во входе (когда $count$ упал в 0) приходится ровно одно «открытие» на выходе. Поэтому $gate$ никогда не превышает 1, что формализовано как *ltl bingate*.

Если ослабить условие до $count>0$ или $count\geq 1$, баланс ломается: лишние $signal(gate)$ поднимут $gate$ до 2 и больше, и *bingate* немедленно даст ошибки. SPIN

использовался для подтверждения, что именно $count=1$ — единственное корректное условие.

5. Формирование LTL-свойств

Для проверки корректности алгоритма сформулированы 4 строгих свойства на языке линейной темпоральной логики (LTL) при помощи вспомогательной переменной `critical`, учитывающей количество процессов в КС.

- Свойство взаимного исключения (Capacity limit):

```
ltl mutex { [] (critical <= K) }
```

Означает, что в любой момент времени число процессов в критической секции не превышает K (в нашем случае 2).

- Бинарность семафора `gate`:

```
ltl bingate { [] (gate <= 1) }
```

Семафор `gate` никогда не превышает значения 1 (строго бинарный).

- Бинарность семафора `S`:

```
ltl binsem { [] (S <= 1) }
```

Семафор `S` никогда не превышает значения 1 (строго бинарный).

- Свойство живости (Progress / No Starvation):

```
ltl liveness { [] <> (critical > 0) }
```

Система не попадает в дедлок (взаимную блокировку), критическая секция бесконечно часто посещается хотя бы одним процессом.

6. Результаты полной верификации в Spin:

Модель успешно проходит проверку всех 4-х свойств:

State-vector 60 byte, depth reached 1806, errors: 0

1716 states, stored

2747 states, matched

4463 transitions (= stored+matched)

13 atomic steps

hash conflicts: 0 (resolved)

```
unreached in proctype process
    main.pml:58, state 34, "-end-"
    (1 of 34 states)
unreached in init
    (0 of 10 states)
unreached in claim mutex
    _spin_nvr.tmp:8, state 10, "-end-"
    (1 of 10 states)
unreached in claim bingate
    _spin_nvr.tmp:17, state 10, "-end-"
    (1 of 10 states)
unreached in claim binsem
    _spin_nvr.tmp:26, state 10, "-end-"
    (1 of 10 states)
unreached in claim liveness
    _spin_nvr.tmp:35, state 10, "(!((critical>0)))"
    _spin_nvr.tmp:37, state 13, "-end-"
    (2 of 13 states)
```

Свойства верифицированы с результатом *errors: 0*.

Статус *unreached in proctype process* подтверждает, что процесс `worker` никогда не достигает своего логического конца (бесконечный цикл).

Статус *unreached in init* подтверждает, что блок `init` выполняется от начала и до конца без каких-либо "зависших" участков.

Статусы *unreached in claim mutex*, *bingate*, *binsem* подтверждают, что финальное состояние автомата-ловушки для инвариантов безопасности является недостижимым. Это значит, что алгоритм ни при каких обстоятельствах не нарушит заданные правила.

Статус *unreached in claim liveness* подтверждает отсутствие дедлоков (deadlocks) и лайвлоков (livelocks).

Выводы.

В ходе выполнения индивидуального домашнего задания был успешно смоделирован алгоритм Барца на языке Promela. Проведена строгая верификация симулятором SPIN. Формальные проверки доказали математическую корректность алгоритма: моделирование одного сложного (счетного) семафора было идеально сведено к операциям над двумя менее мощными - бинарными семафорами и одной целочисленной переменной состояния. Гарантируется выполнение взаимоисключающего доступа к переменной состояния, лимитирование общей емкости (одновременно не более $K = 2$ процессов) и отсутствие взаимных блокировок. Простые атомарные макросы `wait(s)` и `signal(s)` прекрасно продемонстрировали возможности SPIN в симуляции низкоуровневых примитивов операционных систем. Полный исходный код программы см. в Приложении А.

ПРИЛОЖЕНИЕ А

Файл main.pml:

```
// Количество параллельных процессов, конкурирующих за критическую секцию
#define N 4
// Начальное значение обобщенного семафора:
// максимальное число процессов, одновременно допустимых в критическую секцию
#define K 2

// Бинарный семафор S - взаимное исключение при доступе к переменной count
byte S = 1;
// Бинарный семафор gate - блокировка/разблокировка процессов на входе
byte gate = 1;
// Целочисленный компонент симулируемого обобщенного семафора
int count = K;

// Счетчик процессов, находящихся в критической секции (только для верификации)
byte critical = 0;

// Операции над бинарным семафором по типу busy-wait.
// wait(s): атомарно ждем s > 0 и уменьшаем s.
// signal(s): увеличиваем s.
#define wait(s) atomic { s > 0; s-- }
#define signal(s) s++

// Процесс, многократно входящий в критическую секцию через обобщенный семафор,
// симулируемый бинарными S и gate.
proctype process(byte id) {
// Бесконечный цикл: не критическая секция -> вход -> КС -> выход
do
:: true ->
// Некритическая секция
skip;

// --- Симуляция операции wait над обобщенным семафором (p1..p6) ---
wait(gate); // p1
wait(S); // p2
count = count - 1; // p3
if
:: count > 0 -> signal(gate); // p4, p5
:: else -> skip;
fi;
signal(S); // p6

// --- Критическая секция ---
// Вход/выход считаем счетчиком critical; алгоритм Барца гарантирует,
// что одновременно в КС находится не более K процессов.
critical++;
assert(critical <= K);
critical--;

// --- Симуляция операции signal над обобщенным семафором (p7..p11) ---
wait(S); // p7
count = count + 1; // p8
if
:: count == 1 -> signal(gate); // p9, p10
:: else -> skip;
fi;
signal(S); // p11
od;
}
```



```

init {
// Атомарный запуск N конкурирующих процессов
    byte proc_id = 0;
    atomic {
        do
            :: proc_id < N ->
                run process(proc_id);
                proc_id = proc_id + 1;
            :: proc_id == N -> break;
        od;
    }
}

// Свойство безопасности: взаимное исключение.
// Обобщенный семафор с начальным значением K допускает в критическую
// секцию одновременно не более K процессов.
ltl mutex { [] (critical <= K) }

// Свойство безопасности: семафор gate остается бинарным
ltl bingate { [] (gate <= 1) }

// Свойство безопасности: семафор S остается бинарным
ltl binsem { [] (S <= 1) }

// Свойство живости: критическая секция посещается бесконечно часто
// (система не попадает в дедлок и продолжает прогресс)
ltl liveness { [] <> (critical > 0) }

```